SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: **Total Marks: 20**

Code of Conduct:

I POOJASREE B (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: POOJASREE B (192411091)

RUBRICS FOR CALCULATION:

Criteria	Weigh tage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

1: HTTP Response Analysis

A web server returns the following HTTP response header:
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive

Questions:

1. **Identify any incorrect or invalid fields in the header.**
2. **Analyze the issue with Content-Lengthand explain its impact.**
3. **Determine whether the status code matches the response correctness.**
4. **Suggest corrections for the identified errors.**

1. Identify any incorrect or invalid fields in the header.

The main error is:

Content-Length: -150

Content-Length cannot have a negative value. It must be a non-negative integer representing the size of the response body in bytes.

The other fields are generally valid:

- HTTP/1.1 200 OK – Valid status line.
- Date – Valid HTTP date format.
- Content-Type: text/html – Valid.
- Connection: keep-alive – Valid for HTTP/1.1.

2. Analyze the issue with Content-Length and explain its impact.

The value:

Content-Length: -150

is invalid because the response body cannot have a negative size.

The correct value should be:

Content-Length: 150

if the response body contains 150 bytes.

Impact

An invalid Content-Length can cause:

- Incorrect interpretation of the response body.
- Problems in determining where the response ends.
- Client-side parsing errors.
- Connection handling problems.
- Possible HTTP protocol errors.

Therefore, the server must send the actual size of the response body as a non-negative integer.

3. Determine whether the status code matches the response correctness.

The status code is:

200 OK

This indicates that the request was successfully processed and the response is valid.

However, the response contains an invalid header:

Content-Length: -150

Therefore, 200 OK does **not fully match the correctness of the response**.

The server should correct the invalid header before returning a successful response.

However, the 200 OK status itself is appropriate if the requested resource was successfully found and generated. The error is specifically in the response header.

4. Suggest corrections for the identified errors.

The corrected response should be:

HTTP/1.1 200 OK
Date: Wed, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: 150
Connection: keep-alive
Corrections

1. Replace the negative value:

Content-Length: -150

with:

Content-Length: 150

2. Ensure that the value exactly matches the size of the response body in bytes.
3. Keep Content-Type: text/html if the response body contains HTML.
4. Use 200 OK if the request was successfully processed.

Conclusion

The major error in the response is the negative Content-Length value. Since content length must always be zero or a positive integer, it should be replaced with the actual response body size. The 200 OK status is suitable for a successful request, but the invalid header must be corrected for the response to be fully valid.

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
  <head>
  <title>TestPage</title>
</head>
  <body>
  <div>
  <p>HelloWorld
  </div>
 </body>
 </html>
```

Questions:

1. **Identify missing or improperly closed elements.**
2. **Analyze how the browser will interpret this structure.**
3. **Determine the impact on rendering and layout.**
4. **Suggest corrections to make the DOM valid.**

1. Identify missing or improperly closed elements.

The following problems are present:

- The element is not explicitly closed with .
- The <!DOCTYPE html> declaration is missing.

Correct form:

HelloWorld

The <html>, <head>, <title>, <body>, and elements are properly closed.

2. Analyze how the browser will interpret this structure.

Browsers are designed to handle some HTML errors automatically. When the browser encounters the tag while the element is still open, it may automatically close the element.

The browser may internally interpret it approximately as:

```
<div>
   <p>HelloWorld</p>
</div>
```

However, relying on automatic browser correction is not good practice because different browsers or rendering engines may handle invalid markup differently.

3. Determine the impact on rendering and layout.

In this simple example, the page may appear almost normally because browsers automatically correct the missing closing tag.

However, improper HTML can cause:

- Unexpected DOM structure.
- Incorrect CSS selector behavior.
- Unexpected margins and spacing.
- Layout inconsistencies across browsers.
- Problems when JavaScript accesses the DOM.
- Accessibility issues.

For example, the default margin of the element may affect the spacing inside the .

4. Suggest corrections to make the DOM valid.

A corrected and valid HTML structure is:

```
<!DOCTYPE html>
<html lang="en">

<head>
   <meta charset="UTF-8">
   <title>TestPage</title>
```

```
</head>

<body>

  <div>
    <p>HelloWorld</p>

  </div>

</body>

</html>
```

Corrections made:

- Added <!DOCTYPE html>.
- Added lang="en" to the <html> element.
- Added <meta charset="UTF-8">.
- Properly closed the element using .

Thus, explicitly closing all elements and using the correct HTML5 document structure makes the DOM valid and improves consistent rendering.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	**/login?user=admin&pass=1234**	**Visible in URL**	**Low**
POST	**/login**	**Hidden in body**	**Moderate**

Questions:

1. **Interpret the differences in data transmission between GET and POST.**
2. **Analyze which method is more suitable for login and why.**
3. **Identify security risks in using GET for authentication.**
4. **Suggest best practices for secure login implementation.**

1. Interpret the differences in data transmission between GET and POST.

GET
/login?user=admin&pass=1234

- Data is appended to the URL.
- The data is visible in the browser address bar.

- It may be stored in browser history and server logs.
- It is not suitable for transmitting passwords.

POST
/login

The data is sent in the HTTP request body:

user=admin&pass=1234

- Data is not displayed in the URL.
- It is better suited for submitting sensitive form data.
- However, POST alone does **not encrypt** the data.

2. Analyze which method is more suitable for login and why.

POST is more suitable for login authentication.

Reasons:

- Login credentials are not exposed in the URL.
- Passwords are not stored in browser history as part of the URL.
- The data is transmitted in the request body.
- It is more appropriate for operations that submit sensitive data.

However, POST must be used with **HTTPS** to properly protect the credentials during transmission.

3. Identify security risks in using GET for authentication.

Using GET for login creates several risks:

- Username and password are visible in the URL.
- Credentials may be stored in browser history.
- Credentials may be recorded in server logs.
- URLs may be stored by proxy servers.

- URLs may accidentally be shared or bookmarked.
- Sensitive information may be exposed through monitoring systems.

Example:

/login?user=admin&pass=1234

This clearly exposes the password and should never be used for authentication.

4. Suggest best practices for secure login implementation.

The following best practices should be followed:

1. **Use POST** instead of GET for submitting login credentials.
2. **Use HTTPS/TLS** to encrypt data between the browser and server.
3. **Hash passwords securely** using algorithms such as bcrypt or Argon2.
4. **Never store passwords as plain text**.
5. **Use secure cookies** with Secure and HttpOnly attributes.
6. **Implement CSRF protection** for login forms where appropriate.
7. **Use server-side validation** for all login data.
8. **Implement rate limiting** to prevent brute-force attacks.
9. **Use multi-factor authentication (MFA)** for additional security.
10. **Avoid storing passwords in logs or URLs**.

Conclusion

POST is better than GET for login, but POST alone is not sufficient for complete security. A secure login system should use **POST + HTTPS + secure password hashing + secure session management**.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

Questions:

1. **Identify missing closing tags.**
2. **Analyze issues with the <img> tag.**
3. **Determine how browsers will handle these errors.**
4. **Suggest a corrected version of the code.**

1. Identify missing closing tags.

The following closing tags are missing:

</h1>

The corrected elements are:

<h1>Welcome</h1>
This is a paragraph

The <img> element does not require a closing tag in HTML5 because it is a **void element**.

2. Analyze issues with the <img> tag.

The given tag is:

<img src="image.jpg">

The main issue is the missing alt attribute.

A better version is:

<img src="image.jpg" alt="Sample image">
Importance of alt

- Provides alternative text if the image cannot be loaded.
- Helps screen readers describe the image to visually impaired users.
- Improves accessibility.

The src attribute is also a relative path, so image.jpg must exist in the correct folder.

3. Determine how browsers will handle these errors.

Browsers are generally tolerant of missing closing tags.

They may automatically interpret:

```
<h1>Welcome
<p>This is a paragraph
```

approximately as:

```
<h1>Welcome</h1>
<p>This is a paragraph</p>
```

However, relying on automatic correction is not recommended because it may result in:

- Unexpected DOM structure.
- Different rendering behavior.
- CSS layout problems.
- Accessibility issues.

If image.jpg cannot be found, the browser will display a broken image icon or alternative text if the alt attribute is provided.

4. Suggest a corrected version of the code.

```
<!DOCTYPE html>
<html lang="en">

<head>
   <meta charset="UTF-8">
   <meta name="viewport"
        content="width=device-width, initial-scale=1.0">
   <title>Sample</title>
</head>

<body>

   <h1>Welcome</h1>

   <p>This is a paragraph</p>


   <img src="image.jpg"
       alt="Sample image">

</body>

</html>
```

Corrections made:

- Added </h1>.
- Added </p>.
- Added the alt attribute to <img>.
- Added lang="en" for accessibility.
- Added <meta charset="UTF-8">.
- Added the viewport meta tag for responsive design.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. **User enters a URL in the browser**
2. **DNS lookup occurs**
3. **HTTP request is sent**
4. **Server processes request**
5. **Response is returned**
6. **Browser renders page**

Questions:

1. **Interpret each step in the communication flow.**
2. **Identify where delays can occur.**
3. **Analyze the role of DNS in the process.**
4. **Suggest optimizations to improve performance.**

1. Interpret each step in the communication flow.
Step 1: User enters a URL

The user enters a website address, such as:

https://www.example.com

The browser identifies the required domain and resource.

Step 2: DNS lookup occurs

The browser needs the server's IP address. DNS converts the domain name:

www.example.com

into an IP address such as:

192.0.2.1

Step 3: HTTP request is sent

The browser connects to the server and sends an HTTP request, such as:

GET / HTTP/1.1
Host: www.example.com

Step 4: Server processes the request

The server receives the request, finds or generates the required resource, and processes it.

Step 5: Response is returned

The server sends an HTTP response containing a status code, headers, and content.

Example:

HTTP/1.1 200 OK
Content-Type: text/html
Step 6: Browser renders the page

The browser receives the HTML, parses it, requests additional resources such as CSS, JavaScript, and images, and displays the final webpage.

2. Identify where delays can occur.

Delays can occur at several stages:

- **DNS lookup** – Slow DNS resolution.
- **Network connection** – Slow or unstable internet connection.
- **Server processing** – Slow database queries or heavy server-side processing.
- **Data transfer** – Large files take longer to download.
- **Browser rendering** – Complex HTML, CSS, or JavaScript can slow rendering.
- **Additional resource requests** – Many images, CSS files, and scripts increase loading time.

3. Analyze the role of DNS in the process.

DNS acts like the **phonebook of the Internet**.

Users enter easy-to-remember domain names:

www.example.com

But computers communicate using IP addresses.

DNS performs the conversion:

Domain Name
↓
DNS Lookup
↓
IP Address
↓
Server Connection

Without DNS, users would need to remember the numerical IP address of every website.

DNS caching can reduce delay by storing previously resolved domain-to-IP mappings.

4. Suggest optimizations to improve performance.

The following techniques can improve performance:

1. **Use DNS caching** to reduce repeated DNS lookups.
2. **Use a Content Delivery Network (CDN)** to serve content from servers closer to users.
3. **Enable HTTP/2 or HTTP/3** for faster communication.
4. **Compress files** such as HTML, CSS, and JavaScript.
5. **Optimize images** by reducing their file size.
6. **Use browser caching** for static resources.
7. **Minify CSS and JavaScript** files.
8. **Reduce unnecessary HTTP requests**.
9. **Optimize server-side processing and database queries**.
10. **Use lazy loading** for images and other resources.

Conclusion

The communication process begins with URL entry and DNS resolution, followed by an HTTP request and server processing. The server then returns a response, which the browser renders. Performance can be improved by optimizing DNS, network communication, server processing, resource size, and browser rendering.